

Share++

Table des matières

1	Analyse préliminaire.....	4
1.1	Introduction.....	4
1.2	Objectifs.....	4
1.3	Planification initiale.....	5
2	Analyse / Conception.....	6
2.1	Concept.....	6
2.2	Stratégie de test.....	9
2.3	Risques techniques.....	9
2.4	Dossier de conception.....	10
3	Réalisation.....	10
3.1	Environnement.....	10
3.2	Structure du projet.....	11
3.3	Fonctionnalités techniques importantes.....	12
3.4	Installation finale :.....	15
3.5	Modèle des paramètres :.....	15
3.6	Description des tests effectués.....	16
3.7	Erreurs restantes.....	16
3.8	Liste des documents fournis.....	17
4	Conclusions.....	18
4.1	Bilan des fonctionnalités principales.....	18
4.2	Bilan personnel.....	18
5	Annexes.....	20
5.1	Cahier des charges.....	20
	INFORMATIONS GENERALES.....	20
	PROCÉDURE.....	22
	TITRE.....	22
	MATÉRIEL ET LOGICIEL À DISPOSITION.....	22
	PRÉREQUIS.....	22
	DESCRIPTIF DU PROJET.....	23
	LIVRABLES.....	24
	POINTS TECHNIQUES ÉVALUÉS SPÉCIFIQUES AU PROJET.....	25
	VALIDATION.....	26
	HORAIRES.....	27
5.2	Sources – Bibliographie.....	28
5.3	Maquettes.....	28

6	Glossaire.....	31
---	----------------	----

1 Analyse préliminaire

1.1 Introduction

Share++ est une solution de partage de fichiers sur un réseau local. Cette application est réalisée dans le cadre d'un projet informatique en développement d'application au CPNV à Ste-Croix. Ce projet vise principalement à simplifier le partage des fichiers d'utilisateurs entre ses différentes machines, sans avoir à passer par des services de cloud tiers ou l'envoi de pièces jointes lourdes et pénibles pour se transférer des fichiers. Il garantit ainsi une meilleure confidentialité de ses données, ainsi qu'une vitesse de transfert maximale car elle ne dépend que du réseau (pas de OneDrive ou de couche supplémentaires qui ajoutent de la complexité). Aucun travail préalable n'a été effectué sur ce projet spécifique en amont de la période de réalisation.

Contrairement aux informations présentes dans le descriptif du projet du cahier des charges, la partie redirection de port et Internet ne sera pas traitée, en raison des restrictions du CPNV vis-a-vis de la redirection de port.

1.2 Objectifs

Les objectifs principaux de Share++ sont repris directement depuis le cahier des charges :

- Sélectionner un dossier à partager via une interface graphique ou via le menu contextuel Windows (clic droit).
- Démarrer/arrêter un serveur HTTP local.
- Générer un lien d'accès et un QR Code pour faciliter la connexion depuis un autre appareil (PC, tablette, smartphone).
- Afficher en temps réel les logs de connexion (ex. : "Utilisateur X a téléchargé Fichier.pdf").
- Copier automatiquement l'URL du serveur (pour faciliter la configuration de la redirection de port.)

Le serveur génère une page web simple listant les fichiers disponibles, avec des liens de téléchargement directs. L'application est compilée en exécutable autonome, ne nécessitant aucune installation de Python sur la machine cible.

1.3 Planification initiale

Le projet débute le 2 février 2026 et dure jusqu'au 31 mars 2026. Il est découpé en 5 sprints :

	02.02	11.02	18.02	25.02	04.03	11.03	18.03	25.03	31.03
Sprint 1 : Analyse									
Sprint 2 : Maquettes, schémas et conception									
Sprint 3 : Code									
Sprint 4 : Sécurité et intégration système									
Sprint 5 : Tests, Documentation									

La planification à d'abord été ajoutée à l'outil de gestion de projet IceScrum, utilisé en début de projet, avant la migration sur Github Projects.

La planification initiale est également disponible sous forme de note markdown en annexe.

2 Analyse / Conception

2.1 Concept

Le projet repose sur une architecture client-serveur, gérée par une application windows native :

- Backend : Un serveur aiohttp asynchrone gère les requêtes HTTP et le service des fichiers
- Frontend : Une page html listant les fichiers partagés servie lorsque le partage est actif
- Persistence : Les paramètres de l'utilisateur sont stockés dans un fichier JSON
- Application : Une application windows native servant à gérer le serveur et ses paramètres.

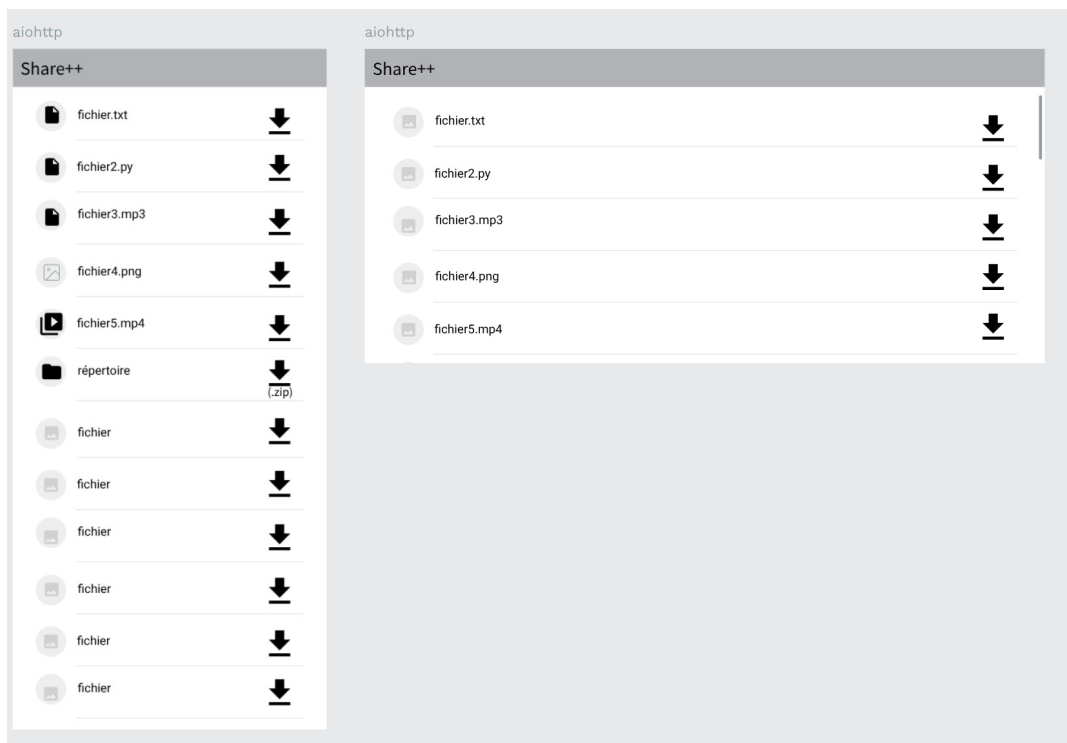
Analyse fonctionnelle : Le cas d'utilisation principal de l'application est de permettre, lorsqu'un utilisateur sélectionne un dossier local, de l'ouvrir via le menu contextuel windows (ou directement depuis l'application), pour ensuite démarrer un serveur de fichier, et de pouvoir y accéder facilement en scannant un code qr généré automatiquement, ou en inscrivant l'url complet dans un navigateur. L'utilisateur pourra ensuite choisir les fichiers ou les répertoires qu'il souhaite télécharger.

Ayant pris connaissance des valeurs à mon sens vertueuse de l'open source et du logiciel libre, j'ai également décidé d'utiliser la licence MIT.

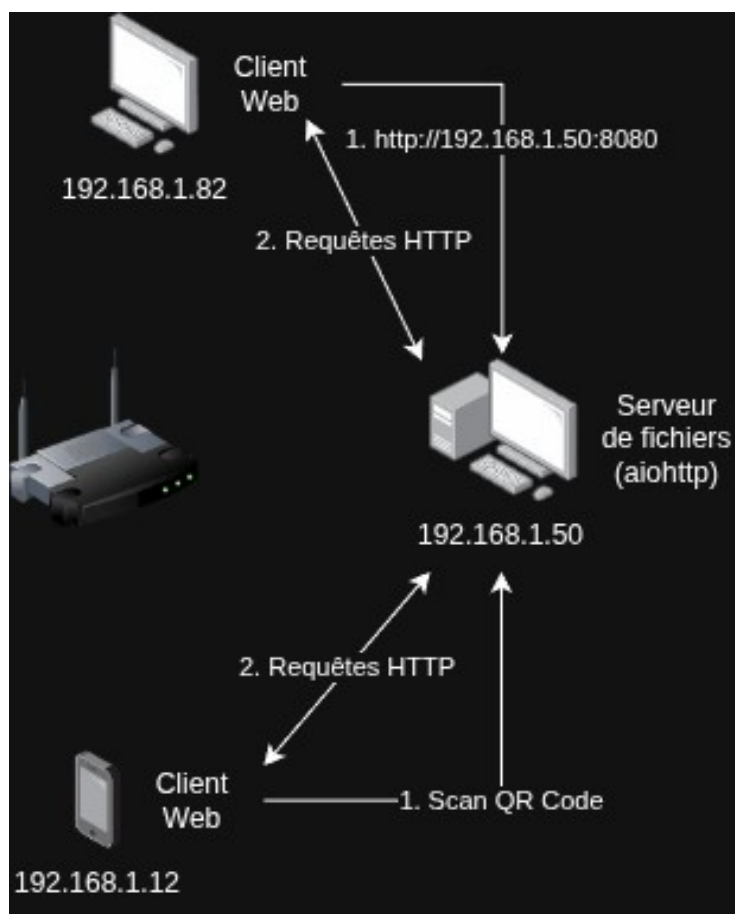
Voici la maquette de l'application



Voici la maquette de l'interface de téléchargement



Voici le schéma réseau :



2.2 Stratégie de test

La stratégie de test vise à valider le fonctionnement des fonctionnalités principales de l'application comme le lancement du service de fichier correctement configuré, ainsi que de garantir une utilisation facile de la solution avec une interface ergonomique.

Types de tests :

1. Tests unitaires : Vérifier l'utilisation du JSON et la génération d'URL
2. Tests d'intégration : Vérifier que le serveur de fichier aiohttp soit bien configuré par rapport aux paramètres rentrés depuis l'application Flet, ainsi que le blocage des attaques traversal.
3. Tests End-to-End : Scan du QR code avec un smartphone, et téléchargement de fichiers
4. Tests d'utilisation : Vérifier l'utilisation fluide et intuitive dans des conditions réalistes.

Moyens de mise en œuvre des tests :

- Automatisation des tests unitaires et d'intégration grâce à la librairie pytest.
- L'environnement de test correspond au matériel utilisé pendant la réalisation : Un laptop personnel Lenovo Yoga sur Fedora 43 (Linux), comme serveur, et un smartphone Android (realme neo7) comme client pour accéder à l'interface de téléchargement.
- Infrastructure réseau : Utilisation du Hotspot du smartphone pour créer un sous-réseau, non-restreint par les restrictions du réseau CPNV.
- Testeurs extérieurs :
 - Nathalie Filipowitz, enseignante de yoga, niveau d'expertise débutant
 - Arnold Filipowitz, actuaire, niveau d'expertise intermédiaire

Couverture et données de test :

Les tests ne sont pas exhaustif. Ils testent les fonctionnalités principales de l'application, et garantissent la sécurité. Les données utilisées seront un fichier .txt pour tester le téléchargement direct, ainsi qu'un répertoire incluant un fichier .txt afin de tester la compression en .zip avant l'envoi.

2.3 Risques techniques

Plusieurs risques ont été identifiés durant la phase d'analyse :

1. **Risque 1** : Le pare-feu Windows pourrait bloquer les requêtes entrantes sur le port 8080
 - Solution : Documenter la notion de port, laisser l'utilisateur spécifier le port s'il le souhaite
2. **Accès simultané** : Plusieurs téléchargement en même temps pourraient faire crasher un serveur pas asynchrone
 - Solution : Utilisation d'une librairie asynchrone (aiohttp) pour gérer plusieurs téléchargements.
3. **Instance multiple** : Si l'utilisateur lance plusieurs fois l'application et qu'il essaie de créer des serveurs simultanément, l'application pourrait crasher.
 - Solution : Implémenter une détection d'instance qui tournerait déjà au lancement de l'application

4. **Sécurité des fichiers** : Risque d'accès non autorisé à des répertoires parents (Directory Traversal attack)
 - Solution : Implémenter une vérification du fichier ou dossier réclamé avant l'envoi

2.4 Dossier de conception

Choix matériel :

La programmation se fera principalement sur un laptop personnel sous Linux.

Choix software :

Le projet est codé en **python (3.14)**, car c'est un langage très utilisé et qui supporte beaucoup de bibliothèques bien maintenues, et qui sont justement utilisés dans ce projet :

- *Flet*, la bibliothèque python cross plateforme pour l'interface de l'application, rendant ainsi possible d'étendre la compatibilité de l'app dans le futur, ainsi que de travailler en utilisant mon laptop personnel pour le développement.
- *aiohttp*, une bibliothèque de serveur HTTP minimaliste et asynchrone pour le serveur de fichier, capable sur le papier de traiter le risque d'accès simultanés au serveur.
- *qrcode*, une bibliothèque python de génération de code QR. Elle est la plus utilisée et est donc bien documentée
- *Boxy SVG*, une application de dessin vectoriel disponible sous Linux. Elle servira à la création de l'icône de l'application.
- *VSCodium*, l'IDE utilisé pour le projet. Une version complètement open-source, de VSCode, compatible avec toutes les extensions VSCode. Il s'agit de l'IDE avec lequel j'ai le plus travaillé durant mon apprentissage.
- *winreg*, une bibliothèque python permettant d'intégrer avec les clés de registres windows, nécessaire à l'ajout du bouton contextuel dans l'explorateur de fichiers sur windows.

Architecture du programme :

J'ai décidé d'adopter une architecture **MVC** (Model View Controller) pour mon projet. Je le trouve adapté à mon projet et pense pouvoir séparer les fonctionnalités comme ceci : Le Modèle gère la persistance des données View contient l'interface de l'application Flet, et de la page web de téléchargement. Controller contient la logique métier de l'application.

Le fichier main.py est utilisé comme point d'entrée et initialise l'application.

Persistance des données :

L'application garde en mémoire les préférences de l'utilisateur via un fichier json. Une solution de base de donnée relationnelle serait à mes yeux une couche de complexité n'apportant pas de réel bénéfices pour l'utilisateur. Les détails du modèle json utilisé sont décrits dans la section 3.4 de ce document.

3 Réalisation

3.1 Environnement

Les détails de l'environnement sont présent en annexe

Développement sur laptop :

J'ai principalement utilisé mon laptop personnel pour réaliser ce travail. J'ai d'abord créé un dépôt sur GitHub : <https://github.com/NathanFilipowicz/Sharepp>, puis j'ai créé des répertoires /src pour inclure les fichiers de code, un dossier /docs pour les documents de projet n'ayant pas d'impact sur le code. J'ai ensuite créé des répertoires correspondant à l'architecture de code MVC (/src/controllers, /src/models, /src/views), ainsi qu'un main.py servant de point d'entrée de l'application flet.

Développement sur ordinateur CPNV :

J'ai eu besoin de migrer le projet sur l'ordinateur fixe du CPNV afin de pouvoir intégrer la fonctionnalité spécifique windows du bouton contextuel de l'explorateur de fichiers. Pour ceci, j'ai cloné le projet depuis mon dépôt github afin de pouvoir travailler sur l'environnement et pouvoir tester l'application directement. J'ai aussi utilisé cet environnement pour créer l'installateur InnoSetup pour le build final windows de l'application.

3.2 Structure du projet

La structure est également représentée plus succinctement dans le README du projet. L'architecture choisie est l'architecture MVC (Model, View, Controller) me permettant de séparer la logique de mon code.

Sharepp/

```
├── docs # Dossier de projet, journaux, maquettes, schémas...
│   ├── Dossier_de_projet_Share++_Nathan2.pdf
│   ├── Dossier_de_projet_Share++_Nathan.odt
│   ├── Dossier_de_projet_Share++_Nathan.pdf
│   ├── Journal_de_bord_Share++_Nathan.pdf
│   ├── Journal_de_test_Share++_Nathan.pdf
│   ├── Journal_de_travail_Share++_Nathan.ods
│   ├── Journaux.pdf
│   ├── maquette_app.pdf
│   ├── maquette_app.png
│   ├── maquette_web.pdf
│   ├── maquette_web.png
│   ├── reseau.drawio
│   ├── schema_reseau.drawio
│   ├── schema_reseau.png
│   └── Sprint_planning.md
├── LICENSE
├── pytest.ini
├── README.md # Instructions d'installation
├── requirements.txt # Dépendances
└── src
    ├── assets
    │   └── icons
    ├── controllers
    │   └── app_controller.py # Logique métier
    ├── main.py # Point d'entrée de l'application
    ├── models
    │   └── model.py # Gestion des données persistentes
```

```
|— report.html
|— settings.json # Paramètres de l'application
|— tests
|   |— test_controller.py
|   |— test_model.py
|— views
|   |— download_view.py # Template HTML pour aiohttp
|   |— view.py # Interface de l'application Flet
```

3.3 Fonctionnalités techniques importantes

Serveur HTTP

- repose sur la librairie aiohttp, très légère, ne bloquant jamais le thread principal de l'application
- asynchrone permettant la gestion multi-clients

L'architecture client-serveur doit se voir ainsi : le serveur écoute sur le port 8080 une fois le serveur lancé depuis l'application. Le client qui accède ensuite à l'adresse IP du serveur sur port 8080, envoie des requêtes GET pour obtenir la liste des fichiers, et pour télécharger un fichier.

Voici le code permettant de démarrer le serveur HTTP, code tiré de la fonction asynchrone `start_server` de mon contrôleur

```
app = web.Application()
# The route for the main page (http://localhost:8080/)
app.router.add_get('/', protected_index)
# The route for file downloads (http://localhost:8080/document.pdf)
app.router.add_get('/{filename}', protected_download)

runner = web.AppRunner(app)
await runner.setup()
site = web.TCPSite(runner, '0.0.0.0', self.model.port)
await site.start()
```

0.0.0.0 permet d'accepter les requêtes depuis toutes les adresses IP du serveur.

J'ai aussi créé une fonction pour la fermeture propre du serveur.

```
async def stop_server(self, e):
    if self.model.server_running and self.model.server_runner:
        await self.model.server_runner.cleanup()
        self.model.server_runner = None
        self.model.server_running = False
        self.view.update_server_status(False)
        self.view.show_controls()
        self.log("Serveur fermé", "error")
```

Gérer proprement le lancement et la fermeture du serveur permet de pouvoir recréer des serveurs de fichiers de manière dynamique et sans restrictions. Ce qui facilite beaucoup l'utilisation de l'application.

Compression des répertoires

- zip des dossiers avant l'envoi

La compression automatique des répertoires est une fonctionnalité pratique pour l'utilisateur qui peut donc télécharger ses répertoires de la même façon qu'il télécharge un fichier quelconque (l'expérience de téléchargement est directe, tout comme un fichier), mais c'est surtout une facilité, car traiter le fait de naviguer chaque répertoire une fois le partage démarré aurait ajouté une importante complexité au projet.

Voici un extrait de la fonction `handle_download`, appelée lorsque le serveur reçoit une requête GET de téléchargement :

```
# Check if the path is a file and is within the allowed directory
if requested_path.is_file() and str(requested_path).startswith(str(base_dir)):
    # Add a header to the response so the downloaded file doesn't just open in the browser
    return web.FileResponse(requested_path, headers={
        'Content-Disposition': f'attachment; filename="{filename}"',
        'Content-Type': 'application/octet-stream'
    })
# Check if the path is a folder
if requested_path.is_dir() and str(requested_path).startswith(str(base_dir)):
    # Create a temporary name for the folder (tmp) to then compress it before sending
    with tempfile.NamedTemporaryFile(delete=False, suffix='.zip') as tmp:
        zip_path = shutil.make_archive(tmp.name.replace('.zip', ''), 'zip', requested_path)
        return web.FileResponse(zip_path, headers={
            'Content-Disposition': f'attachment; filename="{filename}.zip"'
        })
```

Voici mon code permettant de traiter le cas de téléchargement d'un simple fichier, et celui d'un répertoire. Dans le cas d'un téléchargement d'un répertoire, j'utilise un 'tempfile' afin que l'archive créée pour le téléchargement soit automatiquement supprimée après la fermeture de l'application. J'utilise ensuite la librairie 'shutil' afin de compresser le répertoire avant l'envoi.

QR Code

- Génération du QR Code en base 64 pour l'affichage direct sur flet

Pour afficher le QR Code, je récupère d'abord l'adresse IP de la machine en tentant d'envoyer une requête au DNS de google, afin de récupérer l'adresse utilisée.

```
def get_local_ip(self):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        s.connect(('8.8.8.8', 80))
        ip = s.getsockname()[0]
    except Exception:
        ip = None
    finally:
        s.close()
    return ip
```

L'adresse est ensuite utilisée pour construire l'url utilisée, puis, on se sert de cette url pour l'enregistrer dans le presse papier de l'utilisateur au démarrage du serveur, et on l'a passe aussi pour générer le qrcode avec la librairie qrcode.

La fonction `show_qr_code` sert ensuite à remplacer la card contenant les contrôles du partage par le qrcode.

```
local_ip = self.get_local_ip()
if local_ip:
    url = f"http://{local_ip}:{self.model.port}"
    if self.model.copy_to_clipboard:
        await ft.Clipboard().set(url)
        self.view.page.show_dialog(ft.SnackBar("Adresse copiée dans le presse-papier"))

    if self.model.qr_enabled:
        qr_data = self.generate_qr_data_url(local_ip)
        if qr_data:
            self.view.show_qr_code(qr_data, url)
```

Environnement de développement :

- Ordinateurs : Ordinateur fourni par le CPNV (windows 11), laptop personnel
- Système d'exploitation : Windows 11, Fedora 43 (Linux)
- Langage : Python 3.14.2
- Version : 1.0
- Bibliothèques supplémentaires :

Librairie	Version	Utilité
flet	0.80.5	Framework d'interface utilisateur pure python cross-platform
qrcode	8.2	Génération de codes QR pour le partage d'URL vers smartphone
aiohttp	3.13.3	Serveur web asynchrone pour traiter les téléchargements
Pillow	11.3.0	Support pour le traitement d'images sur python
Winreg	3.14.2	Gestion des clés de registres windows pour l'ajout du bouton contextuel dans l'explorateur de fichier

3.4 *Installation finale :*

Cette section décrit comment l'installateur installe ses fichiers sur la machine de l'utilisateur et par quel moyen il a été réalisé.

Le projet passe par **deux** étapes de build.

- D'abord on génère un src.exe avec la commande <flet build windows>. Ceci génère un dossier /build dans /src. Un exécutable est alors généré sous /src/build/windows/src.exe, ainsi que de nombreux fichiers .dll et les répertoires /data, /DLLs, /Lib et /site-packages. Ceci nous permet déjà de transformer l'application en un exécutable, ou plutôt un environnement d'exécution portable.

Mais pour vraiment pouvoir avoir un exécutable installant proprement toutes ces dépendances dans des répertoires du système et proposer la meilleure expérience utilisateur, il faut créer un installateur.

- La deuxième phase du build se passe dans InnoSetup. Il s'agit de créer un assistant d'installation unique, allant ajouter les dépendances de l'application dans les répertoires dédiés sur windows ;
 - C:\Program Files\Share++ (pour toutes les dépendances)
 - C:\Users\<nom_d'utilisateur>\AppData\Roaming\SharePlusPlus (pour les paramètres (settings.json))

Voici le script InnoSetup utilisé

```
[Setup]
AppName=Share++
AppVersion=1.0
DefaultDirName={autopf}\Share++
DefaultGroupName=Share++
UninstallDisplayIcon={app}\src.exe
Compression=lzma
SolidCompression=yes
OutputDir=. \installer_output
OutputBaseFilename=SharePlusPlus_Setup
SetupIconFile=C:\CPNV\Sharepp\src\assets\app_icon_compressed.ico

[Files]
Source: "C:\CPNV\Sharepp\src\build\windows\src.exe"; DestDir: "{app}"; Flags: ignoreversion

Source: "C:\CPNV\Sharepp\src\build\windows\data\*"; DestDir: "{app}\data"; Flags: ignoreversion recursesubdirs createallsubdirs
Source: "C:\CPNV\Sharepp\src\build\windows\DLLs\*"; DestDir: "{app}\DLLs"; Flags: ignoreversion recursesubdirs createallsubdirs
Source: "C:\CPNV\Sharepp\src\build\windows\Lib\*"; DestDir: "{app}\Lib"; Flags: ignoreversion recursesubdirs createallsubdirs
Source: "C:\CPNV\Sharepp\src\build\windows\site-packages\*"; DestDir: "{app}\site-packages"; Flags: ignoreversion recursesubdirs createallsubdirs

Source: "C:\CPNV\Sharepp\src\build\windows\*.dll"; DestDir: "{app}"; Flags: ignoreversion

[Icons]
Name: "{group}\Share++"; Filename: "{app}\src.exe"
Name: "{commondesktop}\Share++"; Filename: "{app}\src.exe"; IconFilename: "{app}\src.exe"
```

3.5 *Modèle des paramètres :*

Le fichier settings.json, situé dans le dossier src/ (puis dans \AppData\Roaming\SharePlusPlus), permet de sauvegarder la configuration de l'utilisateur. Voici sa structure :

```
{
  "selected_path": "/path/to/...", // Chemin du dossier partagé
  "port": <port>,                // Port d'écoute du serveur (défaut=8080)
  "is_protected": true/false,     // Activation de la sécurité
}
```

```
"password": "", // Mot de passe (optionnel)
"logs_visible": true, // État de l'affichage de la console
"qr_enabled": true, // Génération du QR Code activée
"remember_path": false // Mémorisation du chemin au lancement
"copy_to_clipboard": true // Copie automatique de l'ip au démarrage
"context_menu_enabled": true // Ajoute ou enlève le bouton contextuel
}
```

3.6 Description des tests effectués

Cette section détaille la mise en pratique de la stratégie de test. Les premiers tests de l'application ont été réalisés le 10 mars 2026.

Conditions exactes des tests

Pour éviter les restrictions et les problèmes potentiels liés au réseau du CPNV, les tests ont été effectués sur un sous-réseau via partage de connexion du smartphone. Voici les appareils utilisés et leurs utilité :

- Serveur : Laptop Lenovo Yoga (Fedora 43)
- Client : Smartphone Realme Neo7 (Android)

Un environnement précis a été utilisé pour se protéger des problèmes éventuels liés à l'infrastructure réseau restrictive du CPNV. Les détails de l'environnement, le plan des tests, ainsi que des rapports de tests sont présents dans les annexes, ainsi que dans le répertoire /docs du dépôt.

Les tests unitaires et d'intégrations ont été automatisés grâce au framework pytest. Des tests utilisateurs ont aussi été réalisés par mes parents afin de voir les problèmes d'ergonomies et d'utilisation de la solution par des personnes pas informaticiennes.

Un test End-to-End est réalisé en fin de semaine afin de s'assurer de la non régression des fonctionnalités de l'application, ainsi que pour détecter les bugs ajoutés.

3.7 Erreurs restantes

Les fonctionnalités principales de l'application sont fonctionnels comme documenté dans le journal des tests. Tous les problèmes bloquant rencontrés ont été traités, mais des défauts mineurs persistent :

1. Authentification à la page de téléchargements
 - Description : Lors de la connexion à la page, l'utilisateur doit obligatoirement rafraîchir sa page pour accéder à l'authentification en cas de partage protégé par mot de passe. De plus, le champ « username » est présent par défaut et ne peut pas être retiré facilement. Il est ignoré lors de la requête, seul le champ mot de passe est traité.
 - Conséquences sur l'utilisation du produit : L'expérience utilisateur est dégradée par ces actions et réflexions inutiles. L'utilisateur s'attend à ce que cette étape soit plus intuitive.

- Actions envisagées : Remplacer l'implémentation de l'authentification par une page de connexion HTML-CSS classique, utilisant un Token expirable.
- 2. Interface de la page de téléchargements
 - Description : L'interface de téléchargement est trop basique et ne permet aucune autre actions que le téléchargement direct des items présents.
 - Conséquences sur l'utilisation du produit : L'utilisateur peut être frustré de ne pas pouvoir effectuer des actions basiques comme sélectionner plusieurs items avant de lancer le téléchargement, prévisualiser le fichier qu'il s'apprête à télécharger, entrer dans les répertoires présents dans la page, pour ne pas avoir à forcément télécharger le répertoire entier.
 - Actions envisagées : Améliorer l'interface, ainsi que changer les fonctions de logiques afin d'être plus permissives sur certains points (support de téléchargement multiples, sécurité anti accès transversal par l'URL (autoriser la lecture en descendant, mais empêcher de remonter.)
- 3. Lancement depuis le menu contextuel via la version .exe de l'application
 - Description : L'ouverture d'un répertoire avec le menu contextuel de l'explorateur de fichier ne se comporte pas correctement et fait crash l'application
 - Conséquences sur l'utilisation du produit : L'utilisateur ne peut pas utiliser la fonctionnalité principale de l'ouverture directe depuis un répertoire.
 - Actions envisagées : Modifier l'ajout du bouton contextuel pour y traiter le cas de l'ouverture depuis la version exécutable de l'application. Il s'agit d'un problème qui devrait être facilement corrigeable dans une prochaine version.

3.8 Liste des documents fournis

Les éléments suivant constituent la livraison finale de mon projet Share++. Tous les documents sont en version 1.0, version de livraison.

Document	Version	Format	Description
Dossier_de_projet_Share++_Nathan	V1.0	PDF	Document de projet, comprenant l'analyse, la conception et la réalisation du projet. Il permet de sous-traiter la réalisation du projet.
Journal_de_travail	V1.0	PDF	Les détails et horaires de mon travail
Journal_de_bord	V1.0	PDF	Résumés des entretiens de projet
Environnement	V1.0	PDF	Les détails de l'environnement utilisé lors des tests
guide_utilisateur	V1.0	PDF	Manuel d'utilisation
guide_installation	V1.0	PDF	Manuel d'installation
Plan_de_test	V1.0	PDF	Détails de tous les tests prévu dans le projet. La procédure et le résultat attendu y sont clairement inscrit pour chacun d'eux
Journal_de_test	V1.0	PDF	Détails des tests effectués

4 Conclusions

4.1 Bilan des fonctionnalités principales

Le projet Share++ semble répondre à majorité des objectifs fixés dans le cahier des charges. Les fonctionnalités principales ont été implémentées avec succès :

- Application windows permettant de sélectionner un dossier à partage sur une interface graphique
- Démarrage d'un serveur HTTP asynchrone localement
- Génération de l'URL de connexion, ainsi qu'un code QR pour l'accès simplifié pour mobile
- Accès aux fichiers grâce à une page de téléchargement accessible depuis un navigateur, fonctionnant donc sur tous les appareils.
- Affichage des logs de connexions

Globalement, les objectifs principaux du projet sont atteints, avec une application fonctionnelle et utilisable dans un contexte réel.

Objectifs atteints :

- Sélectionner un dossier à partager via une interface graphique ou via le menu contextuel Windows (clic droit).
- Démarrer/arrêter un serveur HTTP local.
- Générer un lien d'accès et un QR Code pour faciliter la connexion depuis un autre appareil (PC, tablette, smartphone).
- Afficher en temps réel les logs de connexion (ex. : "Utilisateur X a téléchargé Fichier.pdf").
- Copier automatiquement l'URL du serveur (pour faciliter la configuration de la redirection de port.)

Points positifs :

- Architecture claire, reprenant le modèle MVC
- Utilisation de technologies modernes et adaptées au projet (aiohttp, flet)
- Interface simple
- Bon fonctionnement des téléchargements en réseau local en termes de performances et de rapidité.
- Gestion multi-clients grâce à asynchrone correctement traitée

Points négatifs :

- L'Interface de l'application peut être amélioré, notamment sur la clarté
- Le contrôleur centralise un peu trop la logique de l'application, il faudrait séparer d'avantage le code en plusieurs fichiers

4.2 Bilan personnel

Ce projet m'a permis d'acquérir de nombreuses notions techniques que j'avais envie d'acquérir, notamment :

- La création de fichiers temporaires supprimés à la fermeture de l'application pour ne pas saturer la machine avec le temps
- La compression d'images pour l'affichage fluide dans l'application

- La création d'un logo grâce a un logiciel de dessin vectoriel (Boxy SVG)
- La programmation d'un logiciel en utilisant un framework « cross-platform », qui permet de se tenir à une intégration intelligente des fonctionnalités par les outils du framework, ou alors de fonctions exécutées qu'en connaissance de l'OS de la machine pour pouvoir continuer à travailler sur ce projet d'application windows, mais sur une machine Linux.
- Le fonctionnement d'un serveur HTTP minimaliste, ainsi que les notions

Je trouve que le code à suffisamment été commenté, et est assez bien structuré pour être repris par la suite.

J'ai également nettement augmenté mes interactions avec les outils de gestion de projet et j'ai également adopté, à mis chemin du projet, suivant également le cours ICT-324 en parallèle, les bonnes pratiques git en terme de commit.

5 Annexes

5.1 Cahier des charges

INFORMATIONS GENERALES

Candidat :	Nom : FILIPOWITZ	Prénom : Nathan
	☐ :	☎ :
Lieu de travail :	☐ CPNV, Rue de la Gare 14, 1450 Sainte-Croix	
Orientation :	☒ 88601 Développement d'application ☐ 88602 Informatique d'entreprise ☐ 88603 Technique des systèmes ☐ 88614 Informaticienne d'entreprise CFC	
Chef de projet :	Nom : Benzonana	Prénom : Pascal
	☐ :	☎ :
Expert 1 :	Nom :	Prénom :
	☐ :	☎ :
Expert 2 :	Nom :	Prénom :

	✉ :	☎ :
Période de réalisation :	Du lundi 2 février 2026 au mercredi 31 mars 2026	
Horaire de travail :	Lundi 08h15-12h30 13h20-15h50 Mardi 08h15-10h50 13h20-16h40 Mercredi 08h15-12h30 13h20-15h50 Jeudi 08h15-12h30 - Vendredi 08h15-11h40 - <i>Toutes les demi-journées ont une pause obligatoire de 15 minutes le matin et de 10 minutes l'après-midi, sauf si elles commencent à 10h05 ou si elles se terminent à 14h55.</i> <i>Les vacances scolaires auront lieu du 12 avril 2025 au 27 avril 2025.</i>	
Nombre d'heures :	90 heures	
Planning (en H ou %)	Analyse : 20%, Implémentation : 50%, Tests 10%, Documentation 20%	
Présentation :	Dates retenues : 27 ou 28 mai 2025	

PROCÉDURE

Le candidat réalise un travail personnel sur la base d'un cahier des charges reçu le 1er jour.

Le cahier des charges est approuvé par les deux experts. Il est en outre présenté, commenté et discuté avec le candidat. Par sa signature, le candidat accepte le travail proposé.

Le candidat a connaissance de la feuille d'appréciation avant de débiter le travail.

Le candidat est entièrement responsable de la sécurité de ses données.

En cas de problèmes graves, le candidat avertit au plus vite les deux experts et son CdP.

Le candidat a la possibilité d'obtenir de l'aide, mais doit le mentionner dans son dossier.

A la fin du délai imparti pour la réalisation du TPI, le candidat doit transmettre par courrier électronique le dossier de projet aux deux experts et au chef de projet. En parallèle, une copie papier du rapport doit être fournie sans délai en trois exemplaires (L'un des deux experts peut demander à ne recevoir que la version électronique du dossier). Cette dernière doit être en tout point identique à la version électronique.

TITRE

Share++ : Solution de partage de fichiers P2P local via HTTP

MATÉRIEL ET LOGICIEL À DISPOSITION

1 ordinateur standard du CPNV avec Windows 10 professionnel et la suite Office 365 et des crédits Azure

Les autres logiciels disponibles sont :

- Windows 11**
- Suite Office**
- Balsamiq**
- IDE (Visual Studio ou PYCharm)**

PRÉREQUIS

Le candidat possède de bonnes connaissances en :

- Programmation Python (asynchrone, gestion de fichiers, sockets).
- Développement d'interfaces graphiques web (via nicegui).
- Intégration système Windows (modification du registre, menu contextuel).
- Gestion de serveurs HTTP légers (via aiohttp).
- Compilation d'applications Python en exécutables autonomes.

DESCRIPTIF DU PROJET

Le projet consiste à développer une application de bureau Windows 11 permettant de transformer n'importe quel dossier local en serveur HTTP accessible via le réseau local (LAN) ou Internet (via redirection de port). L'objectif est de simplifier le partage de fichiers volumineux entre utilisateurs, sans passer par des solutions cloud tierces (OneDrive, SwissTransfer, WeTransfer, etc.), qui impliquent des uploads longs et posent des questions de confidentialité.

L'application permettra à l'utilisateur de :

- Sélectionner un dossier à partager via une interface graphique ou via le menu contextuel Windows (clic droit).
- Démarrer/arrêter un serveur HTTP local.
- Générer un lien d'accès et un QR Code pour faciliter la connexion depuis un autre appareil (PC, tablette, smartphone).
- Afficher en temps réel les logs de connexion (ex. : "Utilisateur X a téléchargé Fichier.pdf").
- Copier automatiquement l'URL du serveur pour faciliter la configuration de la redirection de port.

Le serveur génère une page web simple listant les fichiers disponibles, avec des liens de téléchargement directs. L'application est compilée en exécutable autonome, ne nécessitant aucune installation de Python sur la machine cible.

Interface Graphique :

- Homepage : sélection manuelle du dossier à partager.
- Boutons : "Démarrer le serveur", "Arrêter le serveur".
- Affichage dynamique : IP locale + port utilisé.
- Bouton "Copier le lien" pour faciliter le partage.
- Zone de logs : affichage des événements (connexion, téléchargement, erreur).
- Génération et affichage d'un QR Code pointant vers l'URL du serveur.

Serveur HTTP :

- Hébergement des fichiers du dossier sélectionné via aiohttp.

- Page web générée automatiquement listant les fichiers avec liens de téléchargement.
- Gestion des requêtes HTTP (GET, HEAD).
- Support des téléchargements multiples simultanés.
- Gestion des erreurs (fichier introuvable, accès refusé).

Intégration au système (menu contextuel Windows) :

- Ajout d'une entrée "Partager avec Share++" dans le menu clic droit de l'Explorateur de fichiers.
- Lancement de l'application avec le dossier sélectionné en paramètre.
- Utilisation de winreg pour modifier le registre Windows (clés Shell).

Accessibilité :

- QR Code généré à partir de l'URL du serveur.
- Compatible avec les appareils mobiles (smartphones, tablettes).
- Interface responsive (adaptée aux écrans de tailles variées).

Distribution :

- Compilation en .exe via PyInstaller.
- Création d'un installateur Windows via Inno Setup (avec icône, raccourci, désinstallation).
- Aucune dépendance externe requise sur la machine cible.

LIVRABLES

Le candidat est responsable de livrer à son chef de projet et aux deux experts :

02.02.2026 : une planification initiale contenant la liste des sprints ainsi que les dates et heures des sprints reviews (confirmées avec le PO).

A chaque fin de sprint :

- Un commit signifiant clairement le livrable pour le sprint
- Une documentation en PDF mise à jour
- Un exécutable de l'application quand la réalisation aura débuté

A la fin du TPI, le candidat livrera :

- Les sources et les données sur le dépôt
- La documentation en PDF sur le dépôt
- Une clé USB avec les sources, la documentation et le journal à jour

Chaque vendredi en fin de matinée, le journal de travail devra être transmis par courriel aux experts et au responsable de projet.

POINTS TECHNIQUES ÉVALUÉS SPÉCIFIQUES AU PROJET

La grille d'évaluation définit les critères généraux selon lesquels le travail du candidat sera évalué (documentation, journal de travail, respect des normes, qualité, ...).

En plus de cela, le travail sera évalué sur les 7 points spécifiques suivants (Point A14 à A20) :

1. *Architecture modulaire (séparation des composants : UI, serveur, intégration système).*
2. *Gestion du réseau local (détection IP, port dynamique, redirection).*
3. *Sécurité et robustesse du serveur (gestion des erreurs, accès aux fichiers, logs).*
4. *Intégration système Windows (menu contextuel, registre, persistance).*
5. *Génération de QR Code et accessibilité mobile (fonctionnalité innovante, non vue en cours).*
6. *Compilation et distribution (exécutable autonome, installateur, absence de dépendances).*
7. *Utilisation raisonnée de l'IA (si applicable — à notifier explicitement).*

Remarque ; L'utilisation raisonnée de l'intelligence artificielle doit, impérativement, être notifiée. Si tel n'est pas le cas, l'évaluation du candidat en sera impactée.

VALIDATION

	Lu et approuvé le :	Signature :
Candidat :		
Expert n°1 :		
Expert n° 2 :		
Chef de projet :		

HORAIRES

Voir Intranet

5.2 Sources – Bibliographie

Flet dispose d'une application de démo très pratique lors du développement de l'interface de l'application : <https://flet-controls-gallery.fly.dev/>

- <https://docs.flet.dev/>
- <https://docs.flet.dev/cli/>
- <https://docs.flet.dev/controls/image/>
- https://examples.flet.dev/icons_browser/
- <https://pypi.org/project/qrcode/>
- <https://stackoverflow.com/questions/166506/finding-local-ip-addresses-using-pythons-stdlib>
- <https://docs.aiohttp.org/en/stable/index.html>
- https://docs.aiohttp.org/en/stable/client_advanced.html
- <https://docs.python.org/3/library/winreg.html>

J'ai également utilisé raisonnablement l'intelligence artificielle durant ce projet. J'ai généré une partie du glossaire par Gemini 3 Flash et je l'ai aussi questionnée lorsque je voulais comprendre des concepts en informatique, principalement sur l'asynchrone, la sécurité des mots de passes et les fichiers temporaires. L'utilisation de l'IA a bien été documentée dans le code lui même avec des commentaires.

5.3 Maquettes

Voici les maquettes réalisées sur PenPot



Figure 1: Maquette de l'application

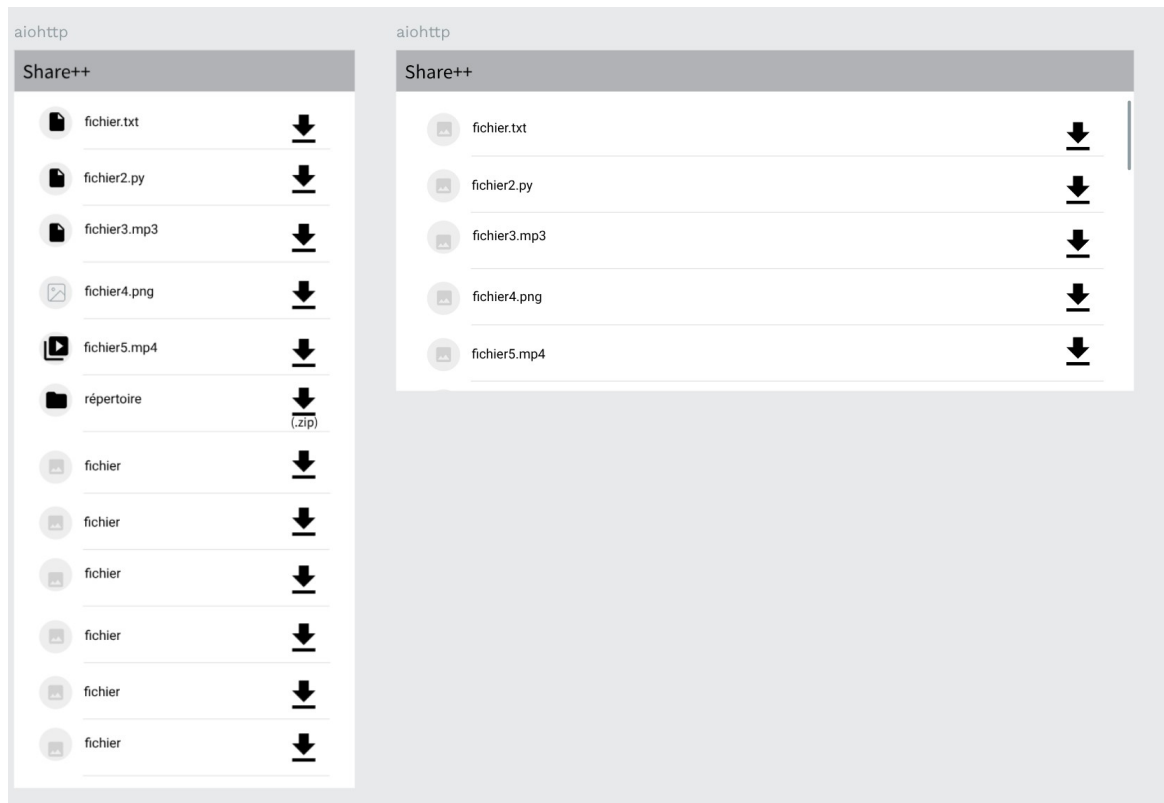


Figure 2: Maquette de l'interface de téléchargement

Voici le logo réalisé sur Boxy SVG

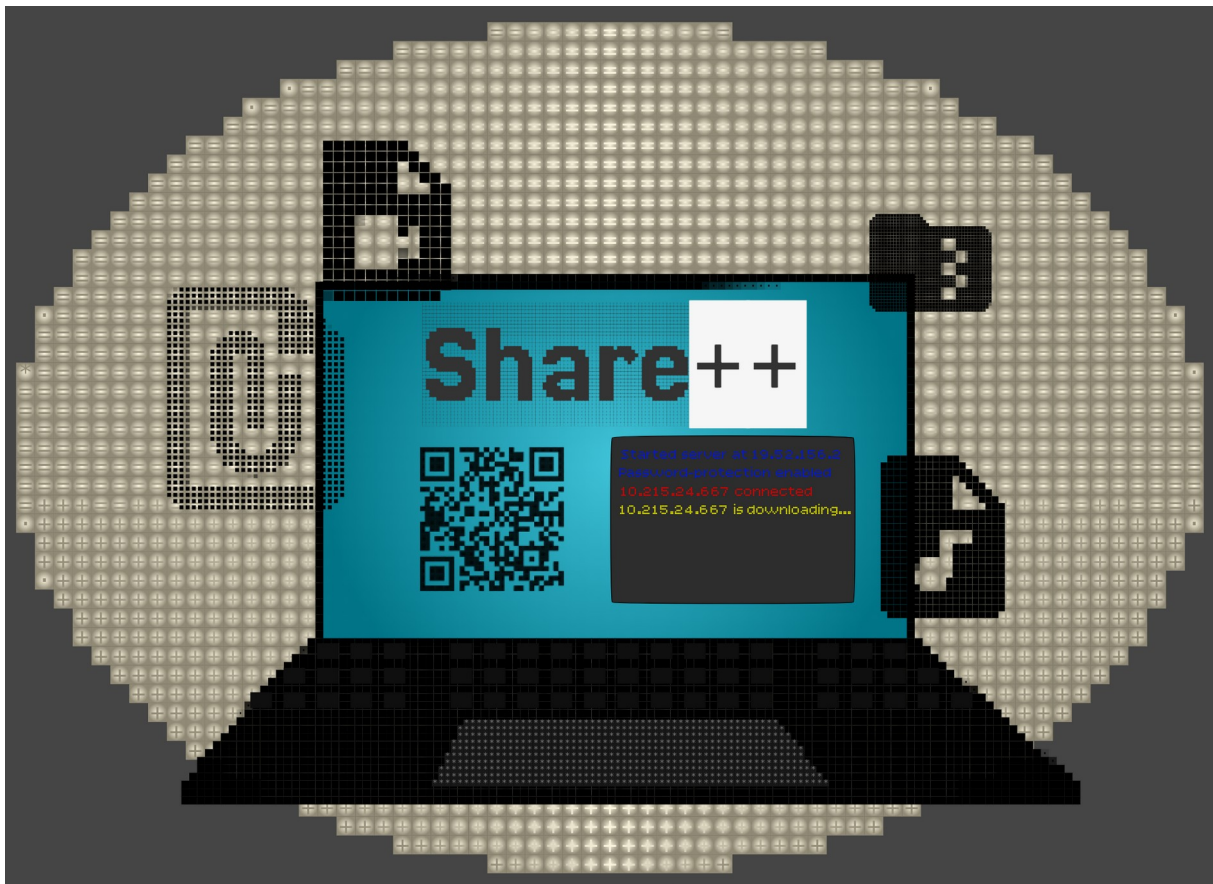


Figure 3: Logo de l'application

6 Glossaire

J'ai généré ce glossaire à l'aide de l'intelligence artificielle (Gemini 3 Flash), tout en l'ayant relu et corrigé :

aiohttp : Bibliothèque Python utilisée pour créer un serveur HTTP asynchrone léger et performant, permettant de gérer plusieurs téléchargements simultanés sans bloquer l'application.

Asynchrone : Mode de programmation permettant d'exécuter plusieurs tâches en parallèle.

Backend : Partie de l'application qui gère la logique métier, le serveur de fichiers et le traitement des données, invisible pour l'utilisateur final.

Cloud : Infrastructures de serveurs mis à disposition par les grands acteurs de l'informatique servant à stocker et rendre des fichiers accessibles en tout temps.

Directory Traversal (Attaque par traversée de répertoire) : Faille de sécurité consistant à tenter d'accéder à des fichiers situés en dehors du dossier partagé.

Flet : Framework Python utilisé pour concevoir l'interface graphique de l'application, permettant un rendu moderne et multiplateforme.

Frontend : Interface visuelle avec laquelle l'utilisateur interagit, comprenant les boutons, les menus et la page web de téléchargement.

InnoSetup : Outil professionnel permettant de créer un programme d'installation (.exe) pour Windows, facilitant le déploiement de l'application chez l'utilisateur.

JSON (JavaScript Object Notation) : Format de fichier texte léger utilisé par Share++ pour sauvegarder de manière persistante les paramètres de l'utilisateur (settings.json).

Machine : Tout type d'ordinateurs d'utilisateurs (PC, Laptop, Smartphone).

MVC (Modèle-Vue-Contrôleur) : Architecture logicielle divisant le code en trois parties distinctes : le Modèle (données), la Vue (interface) et le Contrôleur (logique), pour une meilleure organisation du projet.

Port : Canal de communication spécifique identifié par un numéro permettant au navigateur de se connecter au serveur de fichiers.

QR Code : Code-barres en deux dimensions généré par l'application pour permettre à un utilisateur mobile d'accéder instantanément à l'adresse URL du partage.

Serveur de fichiers : Machine capable de répondre à des requêtes clientes en servant des fichiers via le réseau.

Tempfile : Répertoire temporaire créé automatiquement par le système pour stocker des fichiers temporaires garantissant qu'aucun fichier inutile ne reste sur le disque après utilisation.

Winreg : Bibliothèque permettant à l'application de modifier le registre Windows afin d'ajouter des options dans le menu du clic droit.